and often relationships that link it to others: owns, manages, collaborates with.

Policies depend on these entities to connect rules to reality. Without a consistent representation, signals from PIPs can’t be meaningfully combined. HR might describe an employee as “active,” Finance as “enabled,” and IT as “provisioned.” Unless those differences are normalized into a shared schema, the PDP can’t reliably evaluate a policy condition like:

```
when { principal.employmentStatus == "active" }
```

In theory, the PDP is simply a decision engine that takes a request and context and returns permit or deny. In practice, it usually includes pre-processing logic that gathers and formats the input data.

As shown in figure 12.2, the PDP typically consists of two parts: a lightweight layer of application logic that collects and shapes entities and a policy evaluation engine—such as Cedar—that makes the actual decision. The preprocessing layer retrieves entities and attributes from PIPs, transforms them into the appropriate schema for policy evaluation, and builds the Cedar request object with the principal, action, resource, and context.